Paralelismo

1 - Paralelismo es un concepto asociado con la existencia de múltiples procesadores ejecutando un algoritmo en forma coordinada y cooperante ¿Es acertada dicha definición?

Para mi verdadero... ¿Qué opinan?

2 - ¿Qué significa el modelo de ejecución MIMD que propuso Flynn? De un ejemplo de uso.

Significa Múltiples Instrucciones trabajando sobre Multiple Datos. Un claro ejemplo es un procesador multi-core.

3 - Enumere 3 paradigmas con los que se puede lograr la computación de alto rendimiento.

OpenMP

1 - ¿Cuál es el funcionamiento de OpenMP en relación con sus hilos/procesos y los recursos que comparten?

En OpenMP se trabaja con memoria compartida. Debido a esto, abstrae al programador de tener que definir de forma manual otros métodos para compartir los datos (Todas las PCs manejan una memoria en común). En OpenMP se definen bloques de paralelismo (Usando directivas del compilador) cuyo contenido será ejecutado por todos los hilos creados (Salvo que se indique explícitamente lo contrario con directivas o comandos).

2 - ¿Cuál es la taxonomía Flynn implementada en OpenMP? ¿Como mejoró en las clasificaciones de paralelismo?

La taxonomía Flynn está basado en instrucciones, directivas que provienen de la UC hacia la CPU, y datos, intercambio entre la Memoria y la CPU. En base a esto, hay cuatro tipos de computadoras: SISD, SIMD, MISI, MIMD.

3 - Describa cómo funciona OpenMP.

OpenMP define mecanismos de comunicación entre procesos pesados utilizando funciones de biblioteca y comunicación mediante el protocolo SSH. Permite crear sistemas distribuidos que se comunican utilizando IPC sincrónica o asincrónica, que puede ser Peer to Peer, o Broadcast.

4 - ¿OpenMP sólo puede ejecutarse en servidores distribuidos?

Si bien es común utilizar OpenMP para sistemas distribuidos, comunicados a través de SSH, OpenMP define mecanismos de comunicación para procesos pesados. Por lo tanto, puedo utilizar OpenMP con distintos procesos pesados dentro de una misma máquina, tanto como en máquinas distribuidas.

5 - ¿Cómo se relacionan los tres componentes de OpenMP?

1. Directivas al compilador.
2. Bibliotecas con funciones.
3. Variables de entorno.

Con las **directivas al compilador**, podemos indicar que zona queremos paralelizar y cuantos hilos necesitamos. Dentro de esta región, podemos utilizar las **funciones de la biblioteca** para conocer el thread id, para administrar bloqueos y anidamiento. Finalmente, con las **variables de entorno** podemos controlar cuántos threads simultáneos vamos a spawnear, entre otras acciones, como el tamaño de cada stack.

Otra respuesta

Los tres componentes de OpenMP son: Directivas al compilador, Bibliotecas de funciones y Variables de entorno.

- En el primero damos las órdenes al compilador, dividiendo las funciones en threads, sincronizándolos, etc. y hay que mencionar que se utilizan los recursos de una computadora.
- En el segundo podemos obtener el id del thread, administrar bloqueos y anidamientos.

- Y en el tercero declaramos la cantidad máxima de hilos, el stack, etc.

Estos 3 componentes trabajan en conjunto para brindarnos, simpleza, robustez y nos sea fácil utilizar la interfaz.

6 - ¿Cuál es el funcionamiento de OpenMP en relación con sus hilos y los recursos que comparten?

OpenMP permite paralelizar código utilizando directivas al compilador, funciones de biblioteca y variables de entorno. Dado que utiliza hilos, toda variable global será compartida por todos los hilos.

MPI

1 - Marque la frase que es/son falsa/s:

a. MPI sólo puede ejecutarse en servidores distribuidos.
b. MPI puede ejecutar una o más computadoras.
c. MPI puede ejecutar un hilo en forma distribuida

2 - Describa brevemente cómo funciona MPI

MPI es usado principalmente en entornos distribuidos y se basa en, dicho de cierta forma, utilizar un conjunto (red) de PCs para un funcionamiento en común y paralelizar los distintos procesos que se ordena ejecutar en ella.

Por ejemplo, teniendo una red de 4 computadoras single core, podrían paralelizarse los procesos que se ejecuten en ella de tal forma que pudiera obtenerse un rendimiento mayor que el de una PC con 2 cores y con hiperthreading.

3 - Justificar: Al utilizar MPI, ¿No es preciso aplicar sincronización para garantizar el correcto acceso a la memoria global?

Verdadero. Con el paso de mensajes aplicamos sincronización y mutua exclusión.

Otra respuesta

Al utilizar MPI, se están utilizando procesos pesados separados, y, por lo tanto, no existe tal cosa como “memoria global”, dado que los procesos pesados no comparten memoria (a menos que se utilice un mecanismo para ello). MPI provee un mecanismo de comunicación entre estos procesos pesados, los cuales no es necesario que se sincronicen.

4 - ¿En MPI, tengo que preocuparme por utilizar mecanismos de sincronización?

En MPI no es necesario preocuparse por la sincronización ya que con el paso de mensajes que realiza MPI se está aplicando automáticamente sincronización y mutua exclusión.

5 - Al utilizar MPI ¿Es preciso realizar sincronización para el correcto acceso a la memoria dentro de cada nodo?

No, no es preciso ya que una característica de MPI es utilizar procesos pesados, por lo que cada proceso tiene su memoria haciendo que no sea necesario tener coherencia para acceder a memoria.

6 - Indique la principal ventaja del modelo MPI sobre OpenMP.

La principal ventaja del primero sobre el segundo es que MPI permite crear sistemas distribuidos, esto es, permite tener procesos comunicados que se encuentren en distintas máquinas. De esta manera, no es necesario sincronizar la memoria y se puede realizar un balance de cargas. También sobre la escalabilidad a muy bajo costo y sin límite.

7 - ¿Cuál es la diferencia entre Gather y Scatter?

Ambos son dos métodos de comunicación broadcast que permite la comunicación entre procesos. Su principal diferencia radica en que en el método Gather, muchos procesos envían datos a un sólo proceso, mientras que en Scatter, un sólo proceso envía distintos datos a muchos procesos.

GPU

1 - ¿Qué significa el modelo de ejecución SIMT introducido a partir del GPGPU? De un ejemplo de uso.

El modelo SIMT (Simple Instruction Multiple Threads) implica que una sola instrucción puede actuar sobre uno o más threads (se puede ejecutar una sola instrucción sobre muchos hilos simultáneamente sobre un conjunto de datos).

Ejemplo: Una instrucción de cambio de color a un grupo de threads (Suponiendo que cada uno controle un color) que forman parte de un grupo mayor, podría hacer un cambio en el dibujo original que estuvieran formando. Si enviamos una instrucción de cambio de color, podemos generar un dibujo nuevo en muy poco tiempo y con una sola instrucción.

2 - Se desea procesar en GPU una matriz de 313 columnas por 2012 filas. Si los threads agrupan en 2 dimensiones en el bloque. ¿Cuál es la conformación que maximice la concurrencia, multidimensional si los límites de hardware del GPU son para bloques (512, 512, 64) y para las grillas (1024, 1024, 512)?

1 grilla de 4 bloques con 512 hilos. Necesito poder procesar en la segunda dimensión (Y) 2012 pixeles, y, por lo tanto, con bloques de 512 hilos, voy a requerir 4 bloques (CEILING (2012 / 512) = 4).

3 - Un algoritmo GPU tiene que procesar 5 imágenes de alta resolución 1366x788 píxeles. Si cada thread procesa 1 píxel. ¿Cuál es la conformación, que maximizará la concurrencia multidimensional, si los límites de hardware del GPU son para los bloques (512, 512, 64) y para las grillas (512, 128, 64)?

4 - Justificar: Al programar GPU es necesario de funciones especiales para enviar mensajes con datos entre thread que no comparten memoria.

Verdadero. Acá se puede hablar de las memorias locales y globales.

5 - En GPU enumere tres ventajas que posee la Memoria Global sobre la de registros.

1. Tienen mayor capacidad.
2. Permite comunicar hilos.
3. Me falta una

Otra respuesta

1. La memoria global es la única que permite comunicar hilos de bloques distintos.
2. La memoria global es más grande que la memoria de registros.
3. La memoria global es la única que puede utilizarse para comunicarse con la CPU.

6 - La organización de hilos Kernel, ¿Se hace en bloques o grillas?

Los threads en el modelo CUDA son agrupados en Bloques, los cuales se caracterizan por:

- El tamaño del bloque: cantidad de threads que lo componen. Es determinado por el programador.
- Todos los threads de un bloque se ejecutan sobre el mismo SM.
- Los threads de un bloque comparten la memoria shared, la cual pueden usar como medio de comunicación entre ellos.

Varios bloques forman un Grid y los threads de diferentes bloques de un Grid no se pueden comunicar entre sí, esto permite que el administrador de bloques sea rápido y flexible, no tiene en cuenta el número de SM utilizados para la ejecución del programa.

7 - ¿Por qué se dice que utilizar GPU es comparable con una supercomputadora, pero de bajo costo?
Por el rendimiento que tiene, supera ampliamente el ancho de banda y la memoria que un conjunto de CPUs puede tener.

8 - ¿Cuál es la relación de los hilos que son utilizados en GPU con los de CPU?

9 - En GPU, ¿Por qué es importante racionalizar el uso de la memoria en Registros?

Es importante racionalizar porque el uso de la memoria en registro tiene un más rápido acceso y además cada hilo tiene acceso privado a un sector de ella.

10 - ¿Puede accederse a la memoria global del GPU utilizando la CPU?

Dado que la CPU tiene que leer los resultados de la GPU de algún lado, se utiliza la memoria global (aquella compartida por todos los bloques) de la GPU para esto.

11 - ¿Cuál es la filosofía de la GPU? ¿Cómo se compara con la de CPU?

Comencemos por describir ambas filosofías.

- **CPU**: Busca acelerar las aplicaciones con varios núcleos de ejecución complejos e independientes, buscando una mejora de velocidad.
- **GPU**: Busca mejorar el desempeño de las aplicaciones.

La principal diferencia reside en el hecho de que un aumento de velocidad de la CPU implica un aumento en la performance de todas las aplicaciones del sistema. Un aumento en la cantidad de procesadores implica un aumento en la performance de todas las aplicaciones del sistema.

Mientras que para ver una mejora de desempeño de las aplicaciones utilizando la GPU, no basta con enchufar la placa de video, sino que también es necesario que las aplicaciones realicen llamadas a las rutinas de la GPU, esto es, es necesario que la utilicen activamente.

Otra respuesta

La GPU está pensada para mejorar el desempeño de la aplicación. Es un conjunto de vectores de procesadores simples que ejecutan miles de threads en paralelo. A diferencia de CPU que donde se busca acelerar las aplicaciones con varios núcleos en ejecución y es muy costoso tener cientos de threads ejecutando en paralelo, por una fracción del costo se puede tener GPU y miles de threads en paralelo.

12 - ¿Es posible cambiar la jerarquía de hilos de la GPU una vez que el kernel (subrutina) se encuentra en ejecución?

No es posible, dado que la GPU es tan sólo una arquitectura configurable, esto es, tan sólo es posible configurarla antes de la ejecución, no durante.

13 - V/F: En GPU la organización de hilos kernel se hace en bloques o grillas.

Falso. Los hilos se organizan en bloques, y los bloques se organizan en grillas.

14 - Un programa que suma tres vectores y muestra el resultado por pantalla. ¿Cuál es la secuencia de pasos que realiza si esta utiliza GPU?

Se debe reservar memoria de la GPU (cudamalloc). Luego se debe copiar los tres vectores a esta memoria reservada (cudamemcpy). Luego se debe transferir el kernel (la subrutina) que suma estos tres vectores, y se debe ejecutar los hilos. Finalmente, se debe leer el resultado de la memoria global de la GPU (cudamemcpy), y mostrar el resultado por pantalla. No hay que olvidarse de liberar la memoria reservada (cudafree).

15 - ¿Cuál es la diferencia entre Shared Memory entre CPU y GPU? Ventajas y desventajas de usar Shared Memory en GPU.

Shared Memory en CPU permite compartir memoria entre procesos pesados, o entre hilos. Mientras que en GPU es una de las memorias disponibles de la GPU que permite compartir memoria con los hilos del mismo bloque. Se puede agregar que la CPU no puede acceder a la Shared Memory de ningún bloque de la GPU, y la única manera de comunicar a estos dos procesadores es utilizando la memoria global de la GPU.

Otra respuesta

Respecto a la CPU, tenemos la memoria global, esta es la única a la que la CPU puede acceder y en donde los threads van a leer la información inicial. Esta memoria es lenta y tiene gran capacidad. Luego tenemos la memoria compartida, esta memoria está dentro de cada bloque y se va a repartir entre todos los threads que integran ese bloque. También hay que mencionar a la memoria de registros, la cual es muy rápida, hay una por thread y lo que aquí no entre se envía a la memoria de textura.

CLUSTER

1 - Marque la frase que es/son falsa/s:

a. Los servidores en cluster poseen misma características hardware y distintos SO que unifica un nodo central.
b. Los servidores en cluster poseen un nodo central que coordina los recursos del resto.
c. Los servidores en cluster se consideran homogéneos.

CLOUD COMPUTING

1 - ¿Cuáles son 2 de las principales características que definen al Cloud Computing?

Son muchas…. Puede ser portabilidad (Fácil acceso a todos lados) y almacenamiento externo (No se requiere disponer hardware para almacenar la información).

2 - ¿Por qué es falsa la siguiente frase? “Cloud Computing está formado por procesadores homogéneos y concentrados”

Para el usuario es invisible la arquitectura de Cloud Computing, es todo depende….